1) Find 10 different model providers and their models released in last 12 months

Research period: 4 Nov 2025 – 4 Nov 2026

Below is a comparison of 10 AI model providers and their models, covering your seven requested fields.

Pricing convention: USD per 1 million tokens (M tokens), with input and output prices shown separately. Prices are API list prices, not ChatGPT or other subscription prices.

Cut-off date: This means the model's training-data knowledge cutoff, not the date on which the model was discontinued.

1. Provider and model comparison

Model Provider	Model	Release date	Cut-off date
OpenAI	GPT-5.4	05-Mar-2026	Aug-2025
Anthropic	Claude Opus 4.6	05-Feb-2026	Aug-2025
Google DeepMind	Gemini 3.1 Pro	19-Feb-2026	Jan-2025
Meta	Muse Spark 1.1	09-Jul-2026	Not disclosed
xAI	Grok 4.6	12-Aug-2026	Not disclosed
DeepSeek	DeepSeek V3.2	01-Dec-2025	Mar-2025
Alibaba Cloud	Qwen3.5 Plus	16-Feb-2026	Apr-2025
Mistral AI	Mistral Large 3	02-Dec-2025	Not disclosed
Z.ai	GLM-5.3 Flash	26-Aug-2026	Not disclosed
Amazon	Nova 2 Lite	02-Dec-2025	Oct-2025

2. API token pricing

USD per 1 million tokens. Input and output are charged separately. Prices are indicative standard API rates, before applicable taxes and discounts.

Provider	Model	Input cost	Output cost
OpenAI	GPT-5.4	$2.50	$15.00
Anthropic	Claude Opus 4.6	$5.00	$25.00
Google	Gemini 3.1 Pro	$2.00	$12.00
Meta	Muse Spark 1.1	$1.25	$4.25
xAI	Grok 4.6	$2.00	$6.00
DeepSeek	V3.2	$0.28	$0.42
Alibaba	Qwen3.5 Plus	$0.40	$2.40
Mistral	Large 3	$0.50	$1.50
Z.ai	GLM-5.3 Flash	$0.15	$0.50
Amazon	Nova 2 Lite	$0.30	$2.50

Pricing notes:

- GPT-5.4 and Claude Opus 4.6 have additional pricing rules for long-context requests.
- Gemini 3.1 Pro has different rates above 200K input tokens.
- Qwen pricing shown is the Alibaba Cloud Model Studio rate.
- GLM pricing can vary by inference provider.
- DeepSeek pricing is the standard API rate; cached input is cheaper.

3. Context window, model type and recommended use

Model	Context window	Type	When to use this model
GPT-5.4	1.05M	Proprietary, reasoning, multimodal	Complex coding, software testing, computer-use automation, debugging and enterprise workflows
Claude Opus 4.6	1M (beta)	Proprietary, reasoning, multimodal	Large codebase analysis, autonomous agents, test-script generation and complex debugging
Gemini 3.1 Pro	1M	Proprietary, multimodal reasoning	Large document analysis, image understanding, coding and complex research
Muse Spark 1.1	1.05M	Proprietary, multimodal reasoning	Visual understanding, tool use, computer interaction and agentic tasks
Grok 4.6	500K	Proprietary, reasoning, multimodal	Coding, engineering workflows, tool calling and knowledge work
DeepSeek V3.2	164K	Open-weight, MoE reasoning	Cost-effective coding, mathematical reasoning, test generation and agent workflows
Qwen3.5 Plus	1M	Proprietary, multimodal reasoning	Visual document processing, coding, long-context tasks and AI agents
Mistral Large 3	256K	Open-weight, multimodal MoE	Enterprise applications, document understanding, coding and private deployments
GLM-5.3 Flash	1M	Open-weight, multimodal	Fast coding, visual programming, tool use and cost-efficient agents

Nova 2 Lite	1M	Proprietary, multimodal	Document processing, customer support, image/video analysis and simple automation

4. Which models are suitable for software QA automation?

Considering practical QA activities, the following are particularly interesting:

QA activity	**Recommended model**	**Reason**
Generate test cases from requirements	GPT-5.4	Strong reasoning and structured outputs
Selenium / Playwright code generation	Claude Opus 4.6	Complex coding and debugging
Analyze screenshots and UI changes	Gemini 3.1 Pro	Multimodal understanding
Desktop application automation	GPT-5.4	Native computer-use capabilities
Analyze large application logs	Gemini 3.1 Pro	Large context window
Generate regression test suites	DeepSeek V3.2	Cost-effective reasoning
AI-based locator generation	Claude Opus 4.6	Code and UI analysis
Enterprise/private AI deployment	Mistral Large 3	Open weights and deployment flexibility
Fast test-case generation	GLM-5.3 Flash	Lower token cost
Document-based test automation	Nova 2 Lite	Multimodal document processing

For your Autodesk Inventor + HALOCAD testing environment, I would prioritize GPT-5.4, Claude Opus 4.6, Gemini 3.1 Pro and DeepSeek V3.2 for a comparative proof of concept.

5. Official documentation and API links

Provider	Documentation
OpenAI	GPT-5.4 API documentation
Anthropic	Claude Opus 4.6 documentation
Google	Gemini 3.1 Pro documentation
Meta	Meta Model API
xAI	Grok API documentation
DeepSeek	DeepSeek API documentation
Alibaba	Qwen Model Studio pricing
Mistral	Mistral Large 3 documentation
Z.ai	GLM model catalogue
Amazon	Nova 2 Lite model card

6. Important research considerations

1. Token pricing is subject to change. Verify the provider's pricing page before procurement.
2. Knowledge cut-off dates are not publicly specified for every model. I have marked undisclosed values rather than estimating them.
3. Context window is the maximum combined prompt capacity supported by the model, subject to provider-specific limits.
4. Open-weight does not necessarily mean open-source training data or unrestricted commercial usage.
5. Some prices differ between direct provider APIs and third-party inference platforms.

My recommendation: Use these 10 providers as your initial AI model evaluation shortlist. Run identical QA prompts against each model and compare accuracy, response time, token consumption, and cost.

A useful next step would be to convert this research into an Excel comparison sheet with all 10 providers, filters, source links, and automatic token-cost calculations.

2) Read 2–3 articles about how transformer works?

A **Transformer** is a deep learning architecture that helps AI understand and generate human language. It is the technology behind modern Large Language Models (LLMs), such as ChatGPT, Gemini, Claude, and many others.

The Transformer architecture was introduced in the 2017 research paper *Attention Is All You Need*.

To understand how it works, let's break it into three important concepts:

1. **Vector Embedding** – Converts words into numbers that AI can understand.
2. **Self-Attention Mechanism** – Helps AI understand relationships between words.
3. **Feed Forward Neural Network (FFN)** – Processes the information and learns patterns.

1. Overall Transformer architecture

Imagine you enter this sentence into ChatGPT:

"The software tester found a bug in the application."

The Transformer processes it through several stages.

Input text

"The software tester found a bug"

1. Vector Embedding

Convert tokens into numerical vectors

2. Self-Attention

Understand relationships and context

3. Feed Forward Neural Network

Process and transform information

Output

Predict the next token

These steps are repeated across multiple Transformer layers to build richer contextual representations.

2. Vector Embedding – How AI understands words

Computers don't naturally understand words like humans. They work with numbers.

Vector embedding converts words or tokens into lists of numerical values.

For example:

Word	Illustrative vector
Apple	[0.25, 0.75, 0.12]
Mango	[0.28, 0.70, 0.15]
Car	[0.85, 0.10, 0.45]
Software	[0.55, 0.35, 0.90]

These are simplified example vectors, not actual ChatGPT embeddings.

How does it work?

Multi-Head Attention Explained: Queries, Keys, and Values Made Simple | DigitalOcean

digitalocean.com

1. Input text is split into tokens.
2. Each token is assigned a token ID.
3. The token ID is mapped to a learned embedding vector.
4. Positional information is added so the model can represent token order.

Example:

Input: "I love software testing"

- I → Token ID
- love → Token ID
- software → Token ID
- testing → Token ID

Each token is converted into a numerical vector.

Why is embedding important?

It allows AI to represent similarities and relationships between concepts mathematically. For example, software and testing may have related representations because they frequently appear in similar contexts.

3. Self-Attention Mechanism – How AI understands context

Most important Transformer concept

Self-attention allows the model to understand how each word relates to other words in the same input.

Consider this sentence:

"The tester opened the application because it was crashing."

What does the word **"it"** refer to?

The model needs to connect "it" with "application", rather than "tester".

Self-attention helps it learn this relationship.

How does self-attention work?

Each token embedding is transformed into three vectors:

- **Q – Query:** What information am I looking for?
- **K – Key:** What information can I offer?
- **V – Value:** What information should be passed along?

The Transformer Architecture (V2) - by Damien Benveniste

newsletter.theaiedge.io

Let's understand each step.

Step 1: Generate Q, K and V

For every input token, the model generates three vectors using learned weight matrices.

Step 2: Calculate attention scores

The Query of one token is compared with the Keys of all tokens.

The mathematical formula is:

$$ Attention(Q,K,V)= Softmax\left(\frac{QK^T}{\sqrt{d_k}}\right)V $$

Where:

- Q = Query matrix
- K = Key matrix
- V = Value matrix
- d_k = dimension of key vectors

- Softmax = converts scores into attention weights

Step 3: Calculate attention weights

For illustration, suppose the model processes:

"The tester opened the application because it crashed."

The attention weights for the word "it" might look like this:

Word	Illustrative attention weight
The	0.02
tester	0.05
opened	0.08
application	**0.65**
crashed	0.20

The higher weight indicates stronger attention to "application" in this simplified example.

Step 4: Combine information

The model uses the attention weights to calculate a weighted combination of Value vectors.

The result is a context-aware representation of the token.

What is Multi-Head Attention?

Instead of calculating attention only once, Transformers use multiple attention heads.

From Attention to Architecture: BERT, GPT and Why They’re Different | by Sandra Kimiring | May, 2026 | Medium

medium.com

For example, different attention heads may learn to focus on:

- Grammatical relationships.
- Relationships between subjects and objects.
- Long-distance word relationships.
- Semantic context.

The outputs of the heads are combined to produce the next representation.

Why is self-attention important?

It helps AI understand the context of words rather than treating each word independently.

4. Feed Forward Neural Network (FFN)

Third important concept

After self-attention, the information goes through a Feed Forward Neural Network.

Think of self-attention as gathering relevant information and FFN as processing that information.

How does FFN work?

A typical Transformer FFN contains two linear transformations with a non-linear activation function between them.

From Theory to Code: Step-by-Step Implementation and Code Breakdown of GPT-2 model. | by Vipul Koti | Medium

medium.com

The mathematical representation is:

$$FFN(x)=W_2 \cdot GELU(W_1x+b_1)+b_2$$

Where:

- X = Input representation.
- W1 and W2 = Learned weight matrices.
- b1 and b2 = Bias values.
- GELU = Activation function.

Simple example

Suppose the input sentence is:

"The software tester found a bug."

After self-attention, each token has contextual information.

The FFN processes each token's representation to learn useful patterns and features.

For example, the model can learn patterns related to:

- Software and application concepts.
- Actions such as testing and finding.

- Relationships between bugs and software.
- Language and semantic patterns.

The FFN is applied independently to each token position, using shared weights across positions.

Why is FFN important?

Self-attention mixes information across tokens. FFN transforms the resulting representations and helps the network learn more complex patterns.

5. How these three concepts work together

Concept	Main responsibility	Simple explanation
Vector Embedding	Numerical representation	Understands tokens as numbers
Self-Attention	Context and relationships	Understands which tokens are related
Feed Forward Network	Feature processing	Processes learned representations

There are also two important supporting mechanisms:

- **Positional encoding/embeddings:** Provide information about token order.
- **Residual connections and Layer Normalization:** Help information and gradients flow through deep Transformer networks.

A Transformer repeats attention and feed-forward processing across multiple layers.

For GPT-style models, this is a decoder-only architecture, which predicts the next token using previous tokens.

6. Practical example: How ChatGPT answers a question

Suppose you ask:

"Generate test cases for a login page."

Here is what happens internally:

1. **Embedding:** Converts your question into numerical vectors.
2. **Self-attention:** Understands the relationship between login, test cases, username, password and validation.
3. **FFN:** Processes learned patterns and generates useful representations.

4. **Next-token prediction:** Predicts the next token based on the previous tokens.
5. **Output:** Generates a response containing login test cases.